

Cyclist Wheel Ranging via Ellipse Fitting in Monocular Images

Evan Moncheck

Department of Electrical and Computer Engineering

Northeastern University

Boston, MA, USA

moncheck.e@northeastern.edu

Abstract—This paper presents a monocular computer vision pipeline that detects bicycle wheels in still images and video and estimates camera-to-wheel distance using ellipse geometry. Images are undistorted using calibrated intrinsics, converted to grayscale, Gaussian blurred, and processed with Canny edge detection followed by morphological dilation. A border mask suppresses undistortion artifacts at the image boundary. Edge contours are split at geometric corners, fit individually using the AMS algebraic ellipse estimator, and filtered by size, axis ratio, arc coverage, and Sampson-distance reprojection error. Surviving candidates are clustered via a Union-Find overlap test and pooled into a single refit per group; a size-consistency filter then discards refits implausibly small relative to the largest detection in the frame. Range is recovered from the pinhole model using an orientation-aware effective focal length, and each wheel center is back-projected to metric 3-D camera coordinates. Detected world positions accumulate into a color-coded scatter plot displayed in feet. Both a static image mode and a video mode are implemented, the latter operating on half-resolution frames for efficiency. The system achieves reliable wheel detection and plausible range estimates at distances of approximately 1.5–10 m under typical outdoor lighting.

I. INTRODUCTION

This project is significant because it furthers the development of software that could keep cyclists like myself safe. As a member of the Northeastern University Cycling team, and a competitive cyclist since I was 9 years old, I have put in innumerable hours training on the bike. While many of these hours were spent in the woods on mountain biking trails, a significant amount have also been spent on public roads.

Looking beyond myself, cycling has had a growing presence over the past few years [1]. On public roads, where cyclists are found training, racing, recreating, and commuting, there is always the risk that an accident with a multi-thousand pound vehicle may cause serious injury or worse. Recently, a professional triathlete, Hannah Henry, was struck and killed while out on a training ride [2]. Additionally, in an effort to improve the way our legal system identifies the fault and cause of incidents like these, a law proposed in Colorado requires the driver involved in incidents involving a cyclist to be subject to an impairment test on the spot [3]. While systemic changes like these are certainly necessary, it would be best if these incidents never happened in the first place. There are already some laws in place meant to mitigate these incidents. One

common requirement is that drivers provide 4 feet of distance to cyclists when passing.

Completely avoiding incidents where cars cause cyclists to be injured is a very formidable problem. To narrow the scope within reason, it is useful to examine the ways in which cars are currently being developed with safety and autonomy in mind. Many new vehicles have range sensors designed to identify the size and proximity of entities in the car's environment. These sensors are well suited for avoiding collisions with large objects across low and high visibility environments. However, lidar sensor performance decreases when the size of the object is reduced, and obstacle *identification* is a task lidar sensors are not well suited for. These weaknesses can be supplemented by strengthening the resilience of a computer vision driven approach. Computer vision operates best in high visibility environments. However, by using the known scale and projected geometry of certain objects — such as bike wheels — the relative location between a camera and a cyclist can be estimated. This relation would allow semi-autonomous driving systems to alert a driver when a cyclist is nearby or assist in maintaining a safe passing distance.

II. OBJECTIVES AND SCOPE

A. Project Goals

Level 1 (Project 2): Implement a static-image pipeline that reliably detects visible 700c/29" bicycle wheels and overlays fitted ellipses on detected rim/tire contours. Success criterion: clean ellipse overlays on a self collected dataset of bicycle images, with qualitative confirmation that wheel contours are correctly identified and non-wheel ellipses are rejected.

Level 2 (Final Project): Extend the pipeline with metric range and 3-D position estimation. Using calibrated camera intrinsics, recover camera-to-wheel depth Z and lateral world position X from each accepted ellipse. Annotate each detection with Z and X in the debug overlay. Accumulate world-frame wheel positions into a scatter plot across all processed frames. Implement a video processing mode.

Level 3 (Extra Credit): Add temporal Kalman filter smoothing to stabilize ellipse parameters across video frames. Would include multi wheel temporal tracking and velocity estimations.

B. Assumptions

- Wheels conform to the 700c standard (outer radius $R = 0.350$ m including road tire).
- At least $\sim 5\%$ of the wheel circumference must be visible and unoccluded ($\text{ARC_COVERAGE_MIN} = 0.05$).
- Images are captured with an iPhone 12 Pro at either the $1\times$ (4.2 mm, f/1.6) or $0.5\times$ ultrawide (1.5 mm, f/2.4) lens; calibration matrices are provided per lens.

C. Out of Scope

Items not addressed in the current work: non-700c wheel standards; automatic front/rear wheel disambiguation; and detections outside the physically plausible range window $Z \in [0.5, 10]$ m, which are explicitly discarded by the pipeline.

You could also potentially use this to bypass the "I am not a robot" captcha challenge of identifying squares that contain bicycles but that was not a primary goal.

III. METHODS

A. Camera Calibration

Intrinsics for each lens were recovered from a planar chessboard target (5×3 inner corners, 36 mm square size). Multiple images of the target were captured at varied distances and angles using both the iPhone 12 Pro $1\times$ and $0.5\times$ lenses, and `cv2.calibrateCamera` was used to solve for the 3×3 intrinsic matrix K and radial/tangential distortion coefficients per lens. Results are saved as .npy files and loaded at runtime.

Because calibration and test images may differ in resolution or orientation, an `adapt_camera_matrix` function corrects the loaded matrix before use. It infers the calibration image dimensions from the principal-point coordinates (assuming $c_x = W/2$, $c_y = H/2$), checks whether the calibration and target image agree on portrait vs. landscape orientation, and if not, swaps $f_x \leftrightarrow f_y$ and $c_x \leftrightarrow c_y$. All intrinsic parameters are then scaled linearly to match the actual image resolution. Undistortion remap maps are precomputed once per sequence via `cv2.initUndistortRectifyMap` and applied to each frame with `cv2.remap`.

B. Preprocessing and Edge Detection

After undistortion, each frame is converted to grayscale. A border mask is derived from the grayscale image by thresholding all nonzero pixels to binary and eroding by a 15×15 structuring element. This mask suppresses the black boundary region introduced by undistortion, which would otherwise produce strong spurious edge contours.

The grayscale image is smoothed with a Gaussian blur (kernel size 9, $\sigma = 3$) and passed to Canny edge detection with hysteresis thresholds $t_{\text{low}} = 40$, $t_{\text{high}} = 110$. The resulting binary edge map is dilated by a 3×3 structuring element ($\text{DILATE} = 3$) to thicken edges and close small gaps in rim contours. The border mask is then applied via bitwise AND to remove boundary artifacts.

In video mode, frames are downsampled to 50% of their original resolution ($\text{SCALE} = 0.5$) before the full pipeline runs. The undistortion maps are precomputed once at this

scaled resolution and reused for every frame. All pixel-space thresholds (SEMI_MAJOR_MIN , SEMI_MAJOR_MAX) are scaled by SCALE accordingly.

C. Contour Tracing and Corner-Based Splitting

Contours are extracted from the edge map using `cv2.findContours` with `RETR_EXTERNAL` and `CHAIN_APPROX_NONE`, retaining the full pixel-resolution boundary chain per connected component.

Each contour is split at sharp geometric corners. A window of size $w = \max(8, \lfloor n/25 \rfloor)$ is used at each vertex i to estimate the incoming and outgoing direction vectors, and the turning angle is:

$$\theta_i = \arccos\left(\frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{|\mathbf{v}_1| |\mathbf{v}_2|}\right) \quad (1)$$

Vertices where $\theta_i > 50$ are marked as split points. All turning angles are computed simultaneously via vectorized NumPy operations to avoid an $O(n)$ per-vertex Python loop. Because contours are stored as closed loops in a linear array, the arc segments at the head and tail of the array are geometrically adjacent; they are stitched back together before returning.

This step is the single most impactful preprocessing decision. Without it, spoke lines, chainstay edges, and floor seams merge with the tire arc into large compound contours whose aggregate geometry cannot be approximated by any single ellipse. After splitting, the smooth rim arc is typically isolated as a clean sub-contour suitable for ellipse fitting.

D. Ellipse Fitting and Candidate Filtering

Each sub-contour with at least $\text{MIN_CONTOUR_PTS} = 8$ points is fit using `cv2.fitEllipseAMS`, the Ath–Matthies–Sturm algebraically stable variant of direct least-squares ellipse fitting. AMS produces a more numerically stable result than the original Fitzgibbon formulation at high eccentricities, which is important for strongly foreshortened wheel ellipses. The returned tuple $((c_x, c_y), (w, h), \theta)$ yields $a = \max(w, h)/2$ and $b = \min(w, h)/2$.

Candidates are eliminated by five sequential filters:

- 1) **Axis bounds:** $a \in [30, 600]$ px (scaled in video mode). These bounds correspond to a 700c wheel from approximately 0.5 m to 15 m.
- 2) **Axis ratio:** $b/a \geq 0.4$, rejecting near-edge-on views.
- 3) **Center bounds:** (c_x, c_y) must lie within the image.
- 4) **Arc coverage:** $n_{\text{pts}} / P_{\text{ellipse}} \geq 0.05$, where P_{ellipse} is the ellipse perimeter (Ramanujan approximation). This rejects fits to very short arcs that could correspond to any conic.
- 5) **Reprojection error:** The mean Sampson distance from contour points to the ellipse must satisfy:

$$\bar{d}_{\text{Sampson}} = \frac{1}{n} \sum_{i=1}^n \frac{|f(x_i, y_i)|}{\|\nabla f(x_i, y_i)\|} \leq 15 \text{ px} \quad (2)$$

where $f(x, y) = (x'/a)^2 + (y'/b)^2 - 1$ is the algebraic residual evaluated in the ellipse-aligned frame.

E. Union-Find Overlap Grouping and Refit

A single tire rim generates multiple candidate ellipses because the inner bead and outer sidewall produce two concentric edge rings that independently survive the per-segment filter. A Union-Find structure merges overlapping candidates. Two candidates are considered overlapping if either center lies inside the other's ellipse boundary, or their center-to-center distance is less than $0.5 \cdot \min(a_1, a_2)$. A size guard rejects pairs whose semi-major axes differ by more than 2:1 to prevent spatially coincident but unrelated ellipses from merging.

All contour points from candidates in the same group are pooled into a single point cloud, and `cv2.fitEllipseAMS` is called on the full cloud to produce one refit ellipse per group. Fitting to the merged cloud rather than averaging per-candidate parameters avoids the systematic undersizing that arises when fitting to individual short sub-arcs. The group size is a tunable parameter `OVERSPACE` and set to 2 in my result generation. This excludes ellipses that can not be bound to an overlapping group.

A final size-consistency pass discards any refit whose semi-major axis is below $0.55 \times a_{\max}$ (where $a_{\max}$ is the largest accepted axis in the current frame), removing spurious small detections that survive the per-candidate filters in frames where a large wheel is present.

F. Range and Position Estimation

Under the pinhole model, a circle of radius R in 3-D space at depth Z subtends a semi-major axis of $a = f_{\text{eff}} R / Z$ pixels. Solving for depth:

$$Z = \frac{f_{\text{eff}} \cdot R}{a} \quad (3)$$

where $R = 0.350$ m and f_{eff} is an orientation-aware effective focal length. Because the iPhone sensor has slightly different f_x and f_y values and the ellipse major axis is generally tilted at angle ϕ in the image, the effective focal length along the major-axis direction is:

$$f_{\text{eff}} = \sqrt{(f_x \cos \phi)^2 + (f_y \sin \phi)^2} \quad (4)$$

The angle ϕ is extracted from the fitted ellipse parameters, correcting for the OpenCV convention that the reported angle describes the minor axis when $h > w$ (in which case 90 is added).

The 3-D wheel center position in camera coordinates is recovered by back-projecting the 2-D ellipse center (c_x, c_y) through the calibrated intrinsics using Z from (3):

$$X = \frac{(c_x - p_x) Z}{f_x}, \quad Y = \frac{-(c_y - p_y) Z}{f_y} \quad (5)$$

with X positive right, Y positive up, Z positive forward. Detections outside $Z \in [0.5, 10]$ m are discarded before accumulation.

G. Scatter Plot Visualization

All accepted 3-D wheel positions are accumulated across every frame of the current image sequence or video. After each frame, a white-background scatter panel is rendered with lateral position X (feet) on the horizontal axis and depth Z (feet) on the vertical axis, with larger Z shown higher. Points are colored on a blue-to-red thermal ramp (blue = earliest detection, red = most recent), labeled with their frame index, and annotated with estimated height Y in feet. Grid lines are placed at “nice” tick values derived from the observed data range.

H. Video Mode

`cv2.VideoCapture` reads frames sequentially from `.mp4`, `.mov`, or `.avi` files. Each frame is resized to 50% resolution, processed by the full pipeline, and written to `output.avi` (XVID codec) at the original source frame rate. The undistortion maps and camera matrix are computed once at the scaled resolution and reused across all frames. World-point accumulation is continuous, so the scatter panel reflects the complete cyclist trajectory over the clip.

I. Debug Output Format¹

Each processed image or video frame produces a 2×2 debug grid arranged as follows:

- **Top-left:** Gaussian-blurred grayscale.
- **Top-right:** Dilated Canny edge map; corner split points marked in cyan, accepted contour segments drawn in green.
- **Bottom-left:** Grayscale with ellipse overlays. Individual per-segment candidate ellipses in red; Union-Find refit ellipses in bright green, annotated with $Z = \dots$ m and $X = \dots$ m.
- **Bottom-right:** Accumulated wheel-position scatter plot.

Debug grid outputs saved in results folder with subfolder matching data source. Results folder in repo contains some results I generated but I did not include all for storage management.

IV. RESULTS AND DISCUSSION

A. Achievement Summary

Level 1 was fully achieved. The corner-splitting step proved the highest-impact single addition: without it, nearly all test images fail to produce valid candidates because tire arcs merge with spoke and frame edges. After splitting, the majority of test images yield at least one accepted candidate on the wheel, and the Union-Find refit consolidates inner-bead and outer-sidewall candidates into a single well-positioned ellipse per wheel.

¹Full source code and datasets available at: <https://github.com/moncheck-e/cyclist-wheel-ranging-cv>. To reproduce results, clone the repository and run from the project root: `python scripts/wheel_ranger.py data/<folder> <cam>` where `<cam>` is 1x or 0.5x. Example image run: `python scripts/wheel_ranger.py data/biking_close 0.5x`. Example video run: `python scripts/wheel_ranger.py data/IMG_7537.MOV 1x`.

Level 2 was achieved. Orientation-aware range estimation, 3-D back-projection, scatter-plot accumulation, and a full video processing mode are all implemented and functional.

Level 3 was not completed. Kalman filter smoothing, multi-wheel tracking, and velocity estimations are the primary items for future work. Future work would also see the use of a tripod and precise world truth data collection to further validate ellipse estimation.

B. Detection Performance

Qualitative evaluation was performed on burst datasets from both the $1\times$ and $0.5\times$ lenses, covering distances of approximately 1.5–10m and a range of viewing angles. Under good lighting the pipeline consistently produces a green refit ellipse on the visible wheel. The axis ratio gate ($\text{AXIS_RATIO_MIN} = 0.4$) provides the most effective rejection of non-wheel circular features. The size-consistency pass eliminates small false positive ellipse estimations when a larger wheel is also detected in the same frame. In frames with no large-wheel detection these components can slip through.

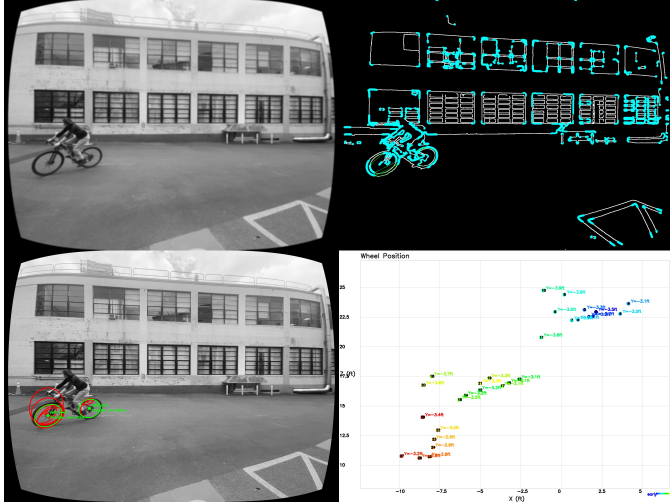


Fig. 1. Level 1 result: static image detection showing (top-left) blurred grayscale input, (top-right) corner-split edge contours, (bottom-left) candidate ellipses in red and Union-Find refit in green, and (bottom-right) accumulated wheel-position scatter plot.

C. Range Estimation

The orientation-aware effective focal length correction is necessary because the iPhone sensor has measurably different f_x and f_y , and wheel ellipses appear at arbitrary tilt angles. Without it, range estimates for oblique ellipses are systematically biased. The primary source of estimation error is ellipse fit quality: when a short arc is visible, the AMS solver is poorly conditioned and can oversize the ellipse, producing a range underestimate. The $Z_{\max} = 10\text{m}$ gate removes the most extreme cases. Because of inconsistencies in over and under-estimation of ellipse sizing, comparison to ground truth was not possible. Qualitative review of range estimates confirmed plausible behavior across the tested distance range.

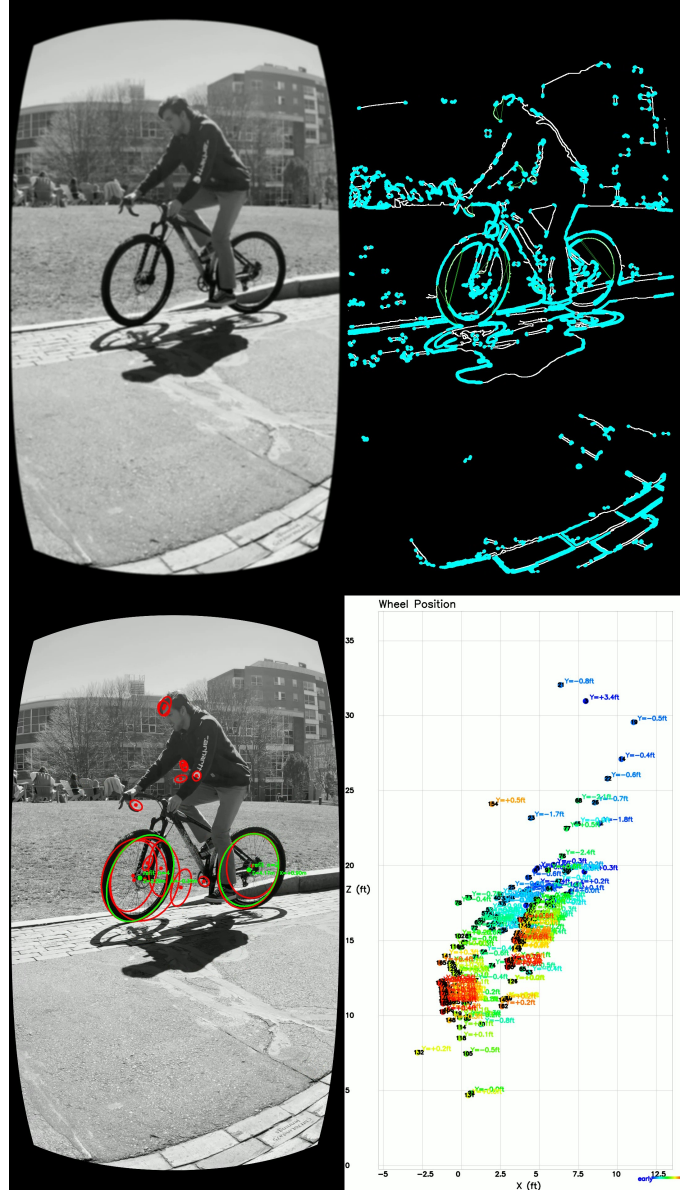


Fig. 2. Level 2 result: video mode output showing ellipse overlays with annotated range and lateral position estimates, and the accumulated scatter plot tracking the cyclist's approach trajectory across frames.

D. Scatter Plot

In burst sequences with consistent detection, the scatter panel shows a coherent approach or lateral-pass trajectory with low point-to-point scatter. The blue-to-red color ramp makes temporal ordering immediately legible. Outlier points from false positives appear as isolated clusters at implausible positions, providing a useful visual diagnostic for parameter tuning.

E. Failure Cases

- 1) **Heavy occlusion:** When more than $\sim 50\%$ of the rim is occluded, no sufficient arc survives filtering and no detection is produced. Alternatively, a poor fit approx-

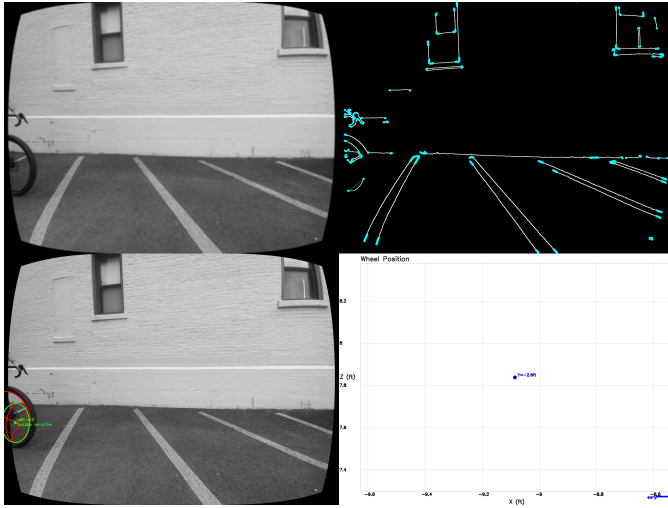


Fig. 3. Example of plot where wheel occlusion results in a poor estimation

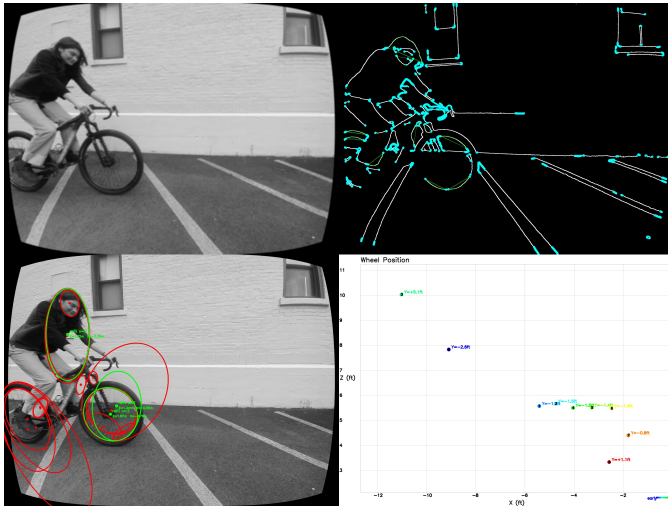


Fig. 4. Example of plot where other curved objects hallucinate a wheel estimation.

imation can be generated when contours from just one side of a wheel are fitted together.

- 2) **Circular clutter:** Other curved objects like people, clothes, handlebars, cables, etc will all generate false positive wheel identification results.

V. CONCLUSION AND FUTURE WORK

A. Conclusions

A monocular bicycle wheel ranging pipeline was designed, implemented, and evaluated. The system integrates per-lens chessboard calibration, portrait/ landscape-aware intrinsic adaptation, Gaussian-Canny-dilation edge detection, vectorized corner-based contour splitting, AMS algebraic ellipse fitting, Union-Find overlap grouping with pooled refit, size-consistency filtering, and an orientation-aware pinhole range formula into a pipeline that processes both still images and video. Corner-based contour splitting was identified as the

critical design decision: virtually all real cyclist images fail without it due to contour contamination from the dense edge structure of a bicycle. The Union-Find refit approach resolves the systematic ellipse undersizing that arises from fitting to short individual tire-bead arcs. The system achieves qualitatively correct range behavior and consistent wheel detection at 1.5–10 m, with a video mode providing practical processing speed through 50% resolution downscaling. By following the color change of the scatter plot, it is possible to reconstruct the path of the cyclist relative the the camera center. An admitted failure of my data collection was not holding the camera steady in any of my recordings so a true comparison to ground truth would require a decomposition of the transform between ground truth and the camera motion.

B. Future Work

Replacing the present per-frame detection with a Kalman filter would provide principled uncertainty propagation and allow track bridging across occluded frames. Integrating a lightweight cyclist detector (e.g., YOLOv8) as a region-of-interest pre-filter would reduce false positives from circular clutter and confine edge computation to cyclist bounding boxes, significantly improving precision. Finally, estimating lateral offset rather than depth range — the quantity directly relevant to the 4-foot law — requires either a stereo baseline or a known ground-plane constraint and is the principal outstanding geometric problem.

ACKNOWLEDGMENTS

The author thanks the EECE 5639 course staff at Northeastern University. The slides relating to Canny edge detection, gaussian blurring, and pinhole camera geometry were most useful to this project. OpenCV and NumPy are used under their respective open-source licenses. Claude was helpful in the debugging process of finding and rectifying sources of error. Thank you to my friends Molly and Michael for being a part of my data sets :).

REFERENCES

- [1] People for Bikes, “Bicycling Participation Report 2024,” <https://www.peopleforbikes.org/news/bicycling-participation-report-2024>
- [2] Triathlon Magazine Canada, “Tragic accident takes the life of Canadian triathlete Hannah Henry,” <https://triathlonmagazine.ca/news/tragic-accident-takes-the-life-of-canadian-triathlete-hannah-henry/>
- [3] Denver7, “Magnus Law moves forward in Colorado Capitol seeking voluntary breathalyzers at serious crash scenes,” <https://www.denver7.com/news/local-news/magnus-law-moves-forward-in-colorado-capitol-seeking-voluntary-breathalyzers-at-serious-crash-scenes/>
- [4] G. Bradski, “The OpenCV Library,” *Dr. Dobbs’ Journal of Software Tools*, 2000.
- [5] C. R. Harris *et al.*, “Array Programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.